



APLICACIONES MÓVILES

DOCENTE: GABRIEL EDUARDO MOREJÓN
LÓPEZ

ACTIVIDAD 2: AUTÓNOMA 1

UNIDAD 1: ARQUITECTURA DE APLICACIONES

VERA DELGADO MICHAEL JOSHUE
BRAVO SOLORZANO JOSÉ RAMÓN
ALVAREZ MOREIRA GABRIEL ELIAS
GUARANDA PEREZ EVELYN BRIGGITTE
QUIROZ AVEIGA KEVIN OSWALDO

28 DE MAY DE 2026

UNIDAD 1: ARQUITECTURA DE APLICACIONES

1. DESARROLLO

Esta actividad tiene como objetivo tomar el prototipo construido en la Actividad 1 y refactorizarlo a un nivel intermedio, organizando la lógica en módulos con responsabilidades claras, aplicando validaciones reutilizables, reglas de negocio simples y documentación técnica suficiente para futuras integraciones con aplicaciones móviles.

El resultado esperado comprende una versión mejorada del prototipo JavaScript con código dividido por responsabilidades, funciones reutilizables de validación, reglas de negocio del módulo claramente implementadas y un documento técnico que explique la estructura, la lógica y las decisiones tomadas.

ENFOQUE ESPECÍFICO DEL MÓDULO EVENTOS ACADÉMICOS

La idea del módulo era construir una herramienta para que la comunidad universitaria pueda registrar y consultar eventos académicos: conferencias, talleres, congresos, seminarios y ese tipo de actividades. Nada muy complejo en cuanto a interfaz, pero sí con una lógica interna ordenada.

En la Actividad 1 el código funcionaba, pero todo estaba concentrado en uno o dos archivos. Para esta segunda entrega el objetivo fue reorganizar eso: separar el almacenamiento, las validaciones, la lógica de negocio y lo que se muestra en pantalla en archivos distintos, cada uno con una sola responsabilidad. El enfoque específico del módulo estuvo en tres cosas: el manejo y validación de fechas de los eventos, la clasificación por tipo o categoría, y el ordenamiento cronológico del listado.

Esa separación también tiene una razón práctica hacia adelante: si en una fase siguiente se quiere conectar el módulo a una API o reutilizar la lógica en una aplicación móvil, los cambios quedan acotados a un solo archivo en lugar de tener que tocar todo.

2. FLUJO PRINCIPAL DEL USUARIO

Al abrir la aplicación por primera vez, el sistema verifica si existen datos guardados en el navegador. Si no hay ningún registro previo, carga automáticamente tres eventos de ejemplo para que el listado no aparezca vacío y el usuario pueda explorar la interfaz de inmediato. Esta carga inicial ocurre en la función `obtenerEventos()` de `storage.js`, que detecta la ausencia de datos y persiste los registros semilla automáticamente.

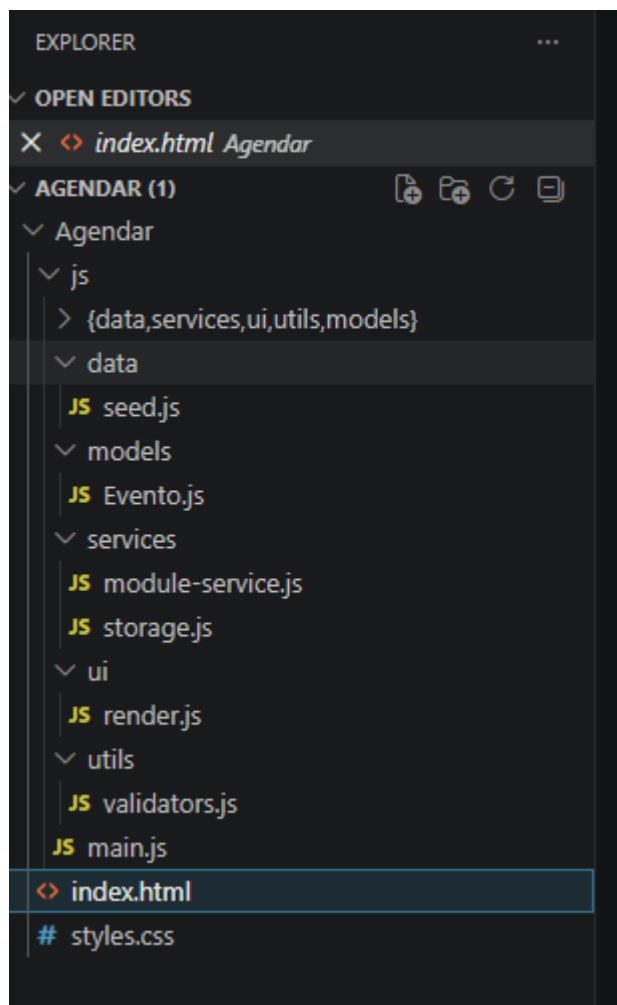
Para registrar un evento el usuario completa el formulario con los campos requeridos: nombre, categoría, fecha, hora, modalidad, ubicación, número de cupos y una descripción opcional. Al presionar el botón de guardar, el sistema valida cada campo de forma individual. Si alguno presenta un problema, aparece un mensaje explicativo directamente debajo de ese campo. Los dos casos de validación más relevantes para el enfoque del módulo son la fecha —que no puede ser anterior al día actual y la capacidad —que debe ser de al menos diez asistentes—. Cuando todos los datos son correctos, el evento se persiste en el navegador y el listado se actualiza de inmediato junto con el contador total.

El listado puede filtrarse combinando búsqueda por texto libre —que revisa nombre y ubicación—, un selector de categoría o tipo de evento y un ordenamiento cronológico por fecha ascendente o descendente. Estas tres opciones funcionan en conjunto y la vista se actualiza en tiempo real. Al pulsar el botón de detalle de cualquier tarjeta, se abre una ventana emergente

con la información completa del evento, que se cierra haciendo clic en el botón de cierre o en cualquier área fuera del cuadro.

3. ESTRUCTURA DE CARPETAS Y ARCHIVOS

El proyecto fue reorganizado siguiendo el principio de separación de responsabilidades. Cada archivo tiene un propósito único y claro, lo que permite localizar y modificar cualquier parte sin afectar al resto:



4. DESCRIPCIÓN DE FUNCIONES PRINCIPALES

EVENTO.JS — EL MODELO

La clase Evento recibe los datos del formulario y construye el objeto. En el constructor pasan tres cosas automáticamente: se asigna el id con `Date.now()`, se genera la fecha de registro en formato legible y el estado se fija como "Pendiente" sin que el usuario pueda cambiarlo. Los cupos se convierten a número entero ahí mismo, así no llegan como cadena de texto desde el formulario.

`hayCupos()` devuelve true si los cupos son mayores a cero. `obtenerResumen()` retorna el nombre del evento con su fecha y hora en una sola cadena, útil para logs o para una vista compacta en una app móvil.

STORAGE.JS PERSISTENCIA

obtenerEventos() lee los eventos de localStorage. Si no hay ninguno, carga el seed automáticamente y lo persiste. guardarEventos() reemplaza la lista completa. agregarEvento() agrega un evento al final y guarda. El archivo también tiene actualizarEvento() y eliminarEvento() que no se usan en esta entrega pero quedan disponibles para fases siguientes.

MODULE-SERVICE.JS LÓGICA DE NEGOCIO

Acá están las funciones más importantes para el enfoque del módulo. fechaValida() verifica que la fecha del evento no sea anterior a hoy. La comparación se hace a las 00:00:00 horas, así que el día actual siempre es válido. ordenarEventos() ordena la lista por fecha —ascendente o descendente— sin modificar el arreglo original, devolviendo una copia ordenada.

crearEvento() recibe los datos del formulario y devuelve una instancia de Evento lista para guardar. cuposValidos() confirma que la capacidad sea de al menos diez. buscarEventos() filtra por texto en nombre y ubicación sin distinción de mayúsculas. filtrarPorCategoria() devuelve todos los eventos o solo los del tipo seleccionado.

VALIDATORS.JS VALIDACIONES

validarFormulario() revisa todos los campos del formulario y devuelve un objeto con los errores por campo, o null si todo está bien. Usa cuatro funciones más pequeñas: campoVacio(), longitudMinima(), numeroMinimo() y fechaNoAnterior(). Cada una hace una sola cosa y puede reutilizarse en cualquier formulario del proyecto sin tocar nada.

RENDER.JS INTERFAZ

renderizarEventos() genera las tarjetas y las inserta en el contenedor. Si la lista está vacía muestra un estado visual con ícono y mensaje. mostrarErrores() limpia los mensajes anteriores y pone el mensaje correspondiente debajo de cada campo que falló. abrirModal() rellena el modal con los datos del evento. mostrarMensajeExito() muestra una confirmación que desaparece sola a los tres segundos.

MAIN.JS PUNTO DE ENTRADA

No tiene lógica de negocio. Solo conecta los eventos del DOM con los servicios y la UI. La función interna refrescarVista() aplica en secuencia la búsqueda, el filtro de categoría y el ordenamiento, y actualiza el listado y el contador. Se llama en la carga inicial y cada vez que el usuario toca algún control de filtro.

5. REGLAS DE NEGOCIO IMPLEMENTADAS

Se implementaron tres reglas propias del módulo. Las dos primeras responden directamente al enfoque específico de la actividad.

REGLA 1 LA FECHA NO PUEDE SER ANTERIOR AL DÍA ACTUAL

Esta es la regla más relevante dado el enfoque del módulo en el manejo de fechas. No se puede registrar un evento que ya pasó. La comparación se hace contra la fecha actual a las 00:00:00 horas para que el día de hoy siempre sea válido.

Está implementada en dos lugares: en fechaValida() dentro de module-service.js para que el servicio pueda verificarla independientemente del formulario, y en fechaNoAnterior() dentro de validators.js para el control de la interfaz. El mensaje que ve el usuario es: "La fecha no puede ser anterior al día actual."

REGLA 2 CAPACIDAD MÍNIMA DE DIEZ ASISTENTES

Un evento con menos de diez cupos no tiene sentido como evento académico formal. La validación opera igual que la de fecha: en `cuposValidos()` dentro de `module-service.js` y en `numeroMinimo()` dentro de `validators.js`. Así la regla es verificable desde la interfaz y desde cualquier integración externa.

REGLA 3 ESTADO INICIAL SIEMPRE "PENDIENTE"

Todo evento nuevo entra al sistema con el estado "Pendiente", sin opción de cambiarlo durante el registro. Eso pasa en el constructor de `Evento`, así que no importa cómo se llame al constructor: el estado inicial siempre es el mismo. En fases siguientes se podría agregar una función para cambiarlo a "Activo" o "Finalizado" con transiciones controladas.

6. DIFICULTADES ENCONTRADAS Y SOLUCIONES

Los nombres de los campos no coincidían con el constructor. El formulario HTML usaba "tipo", "lugar" y "capacidad", pero el constructor de `Evento` esperaba "categoria", "ubicacion" y "cupos". Al principio el evento se creaba con campos vacíos sin que saltara ningún error. Se revisaron todos los campos uno por uno, se alinearon los nombres y se dejó documentada la correspondencia en los comentarios del código.

LA LÓGICA DE VALIDACIÓN ESTABA REPETIDA EN DOS ARCHIVOS

`validators.js` y `module-service.js` tenían funciones que hacían exactamente lo mismo. El problema era que no estaba claro qué responsabilidad tenía cada uno. Se definió que `validators.js` valida el formulario y devuelve errores por campo, y `module-service.js` tiene las funciones de negocio para uso interno del servicio. Esa distinción resolvió la duplicación.

SEED.JS ESTABA VACÍO

Al abrir la aplicación por primera vez el listado aparecía completamente en blanco. `seed.js` no tenía ningún evento. Se agregaron tres eventos académicos representativos y se modificó `obtenerEventos()` para cargarlos automáticamente si `localStorage` no tiene datos.

LOS ERRORES DE VALIDACIÓN USABAN `ALERT()`

Cada vez que el formulario fallaba, aparecía un `alert` que interrumpía la página y no indicaba en cuál campo estaba el error. Se reemplazó por `mostrarErrores()` en `render.js`, que inyecta el mensaje debajo del campo específico que no pasó la validación.

INCONSISTENCIA EN LAS CLASES CSS PARA OCULTAR ELEMENTOS

Algunos archivos usaban "hidden" y otros "oculto" para lo mismo. Eso hacía que ciertos elementos no se mostraran o no se ocultaran correctamente. Se estandarizó todo el proyecto para usar solo "oculto".

7. MEJORAS PROPUESTAS PARA FASES POSTERIORES

Durante el desarrollo quedaron pendientes varias cosas que no entraban en el alcance de esta entrega pero que tienen sentido para una siguiente versión:

- **Conexión con una API.** El módulo usa `localStorage`, lo que significa que los datos solo existen en el navegador donde se registraron. Reemplazar `storage.js` por llamadas a una API REST sería el cambio más importante para una versión real, y la arquitectura actual lo hace posible sin tocar los otros archivos.

- **Reutilización en una app móvil.** Los módulos `module-service.js` y `validators.js` no tienen dependencias del DOM, así que podrían usarse directamente en un proyecto React Native.
- **Gestión de estados del evento.** Actualmente el estado es siempre "Pendiente". Faltaría una función para cambiarlo a "Activo" o "Finalizado", con reglas que eviten transiciones inválidas —por ejemplo, no poder volver de "Finalizado" a "Pendiente"—.
- **Sistema de inscripciones.** El campo cupos ya está implementado. Sería natural agregar una lista de inscritos por evento y que los cupos se descuenten en tiempo real.
- **Paginación.** Con pocos eventos el listado funciona bien, pero si crece se vuelve incómodo. Agregar paginación o carga progresiva resolvería eso.
- **Exportación.** Una opción para descargar los eventos en CSV o PDF facilitaría el trabajo administrativo sin necesidad de copiar datos a mano.

8. COMPARACIÓN CON EL PROTOTIPO ANTERIOR

La siguiente tabla muestra los cambios concretos entre lo entregado en la Actividad 1 y esta versión refactorizada:

Aspecto	Antes → Ahora
Archivos JS	2 archivos → 7 archivos con responsabilidades separadas
Organización del código	Todo en <code>app.js</code> → dividido en <code>model</code> , <code>services</code> , <code>utils</code> y <code>ui</code>
Campos del evento	nombre, tipo, fecha, lugar, capacidad → + hora, modalidad, descripción, cupos
Modelo de entidad	Objeto literal → clase <code>Evento</code> con constructor y métodos
Validación de fecha	Sin validación → <code>fechaValida()</code> con comparación a 00:00 h
Tipos de evento	Sin filtro → campo categoría + filtro en tiempo real
Orden cronológico	Sin ordenamiento → <code>ordenarEventos()</code> asc/desc sin mutar el arreglo
Mensajes de error	<code>alert()</code> → mensaje debajo de cada campo con error
Reglas de negocio	Implícitas → explícitas en <code>module-service.js</code>
Datos de ejemplo	Arreglo vacío → tres eventos cargados automáticamente
Estado vacío	Texto plano → bloque visual con ícono
Confirmación al guardar	Ninguna → mensaje que desaparece solo
Contador de eventos	No existía → badge actualizado en tiempo real

Ejecución

Gestión de Eventos Académicos

Registrar Nuevo Evento

Nombre del Evento:

Categoría:

Fecha:

Hora:

Modalidad:

Lugar / Ubicación:

Capacidad (mín. 10 asistentes):

Descripción:

[Guardar Evento](#)

Listado de Eventos

Total: 4

Congreso Internacional de Ingeniería de Sistemas

Categoría: Congreso

Fecha: 15 de septiembre de 2025 — 09:00

Modalidad: Presencial

Ubicación: Auditorio Central, Bloque A

Cupos: 200

[Pendiente](#) [Ver detalle](#)

Taller de Desarrollo Web con React

Categoría: Taller

Fecha: 20 de septiembre de 2025 — 14:00

Modalidad: Virtual

Ubicación: Plataforma Teams

Cupos: 40

[Pendiente](#) [Ver detalle](#)

Seminario de Ética en Inteligencia Artificial

Categoría: Seminario

Fecha: 5 de octubre de 2025 — 10:30

Modalidad: Híbrido

Ubicación: Sala de Conferencias, Bloque B

Cupos: 80

[Pendiente](#) [Ver detalle](#)

FCI ANIVERSARIO

Categoría: Conferencia

Fecha: 14 de octubre de 2026 — 11:11

Modalidad: Híbrido

Ubicación: Auditorio

Cupos: 85

[Pendiente](#) [Ver detalle](#)